

claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_3d536535-241\agent-a40bb13516c4e59a2.jsonl`  
- SHA-256: `2736a20f236151b27ac047b0097b2b3c877cbc5871b4a457d22d5f0d006ef451`  
- Source modified: `2026-06-16T03:25:58+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_3d536535-241`  
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`
```

Transcript

1. user (2026-06-16T03:24:28.706Z)

Adversarial Claim Verifier (voter 2/3)

Be SKEPTICAL. Try to REFUTE this claim. 2/3 refutations kill it.

Research question

Research question: **What concrete, recent (2022-2025) techniques and evaluation practices should a local, commercially-licensed badminton rally-detector adopt to close the quality gap with a strong cloud VLM (Gemini), given a tiny labeled set (~6 - ~16 golden videos)?** Produce an adversarially-verified, cited report organized around the four dimensions below. Prefer specific named methods, papers (arXiv/venue), and reported numbers over generic advice; flag anything you cannot verify.

CONTEXT (so you target the real gap, not generic ML):

- System: a local pipeline detects the shuttle per-frame (WASB / TrackNet, MIT-licensed) ? a heuristic turns the trajectory into candidate rally windows ? today an optional Gemini "handoff" confirms rallies. We just measured a fully-local (no-AI) run vs Gemini on 3 matches: local emits far FEWER but MUCH LONGER windows (mean 65s vs 6.1s; one window spanned a whole 174s clip) ? it does NOT miss activity, it FAILS TO SEGMENT. Root cause (code-confirmed): a frame is "active" iff the shuttle is visible AND moving above a small per-second-normalized velocity threshold; active frames within a 2 s gap are merged; there is NO max-window cap and NO inactivity/boundary split. So "shuttle is moving" is conflated with "rally in progress," and dead-time + serves + knock-ups bridge rallies into mega-windows. Shuttle tracking itself is excellent (96-99% per-frame visibility).

- HARD CONSTRAINTS: weights/data/code must be MIT/Apache-2.0/BSD only (no viral/NC/custom licenses, reject Gemma-3/Llama-4-MAU-cap); we may NEVER train owned weights on paid-LLM (Gemini/OpenAI/Anthropic) outputs (terms/copyright), though paid LLMs may be used at inference; minors excluded from any training pool; runs on a single local GPU.

- WHAT WE ALREADY HAVE (do NOT re-survey the basics ? go deeper/newer): the architecture skeleton is known to be standard Temporal Action Localization/Segmentation (proposal+scorer) + late/score-level fusion + stacked generalization; in-domain prior art (MonoTrack, ShuttleSet, See et al. 2024/25 non-broadcast badminton rally start/end, Ghosh et al. point-segmentation, tennis/TT play-break state machines) is already cited. We have a working eval harness: leave-one-video-out (grouped by match), temporal-IoU F1 @0.5, F1@{0.1,0.25,0.5}, mAP@tIoU, segment-count-ratio, bootstrap 95% CIs, paired exact sign-test, Platt/isotonic calibration + Brier/ECE, a tiny numpy LogisticRegression scorer over per-window feature columns, an experiment leaderboard, and an "eval+serve" guard (a cue measured +0.074 F1 on permissive proposal windowing but ?0.008 on the coarser served windowing ? reverted). Available but default-OFF per-window signals: net-crossing count (rally gate), 5 person/court features, 3 audio-hit features.

THE FOUR DIMENSIONS:

1) WINDOWING & BOUNDARIES (Temporal Action Localization/Segmentation): beyond proposal+score ? what are the best **recent** methods for (a) turning a dense per-frame signal into well-segmented

actions, (b) explicitly controlling OVER-segmentation/merging, (c) boundary detection/regression and splitting merged segments? Cover e.g. MS-TCN/MS-TCN++, ASRF (action-boundary regression), BMN/BSN boundary-matching, ActionFormer/TriDet/TemporalMaxer (actionness + boundary heads), smoothing/boundary losses, and the standard metrics for over-segmentation (segmental F1@k, edit/Levenshtein score, mAP@tIoU). Which transfer to a 1-D shuttle-trajectory + auxiliary-cue setting on tiny data?

2) SHUTTLE-TRAJECTORY ? RALLY SIGNAL PROCESSING (racket-sports specific): how do published systems convert ball/shuttle tracking into rally/point segments and distinguish "in-play" from dead-time? Cover rally-state machines, serve/let detection, net-crossing & two-sided-occupancy cues, hit/contact (audio + kinematic) detection, and fps-invariant trajectory features. What features best separate "rally" from "shuttle merely moving"?

3) SMALL-DATA STRATEGIES: with ~6?16 labeled videos, what gives the most quality per label ? active learning (which videos/segments to label next; uncertainty/diversity/core-set), semi-/weakly-/point-supervised TAL (e.g. SF-Net, timestamp supervision), self-training/pseudo-labeling, and data-efficient TAL? Separately: the legally-clean ways to exploit a strong teacher (Gemini) at INFERENCE for weak/pseudo-labels or as an evaluation oracle WITHOUT its outputs becoming training data for owned weights ? what does the literature say about teacher-student / distillation boundaries and label-noise handling, and what governance distinctions matter?

4) EVALUATION METHODOLOGY & RIGOR at small n: trustworthy measurement with ~6?16 videos ? bootstrap/permutation tests, paired sign-tests, leave-one-group-out vs RL00CV bias, calibration (reliability, ECE), the over-segmentation-blindness of plain tIoU, shadow/A-B evaluation, regression gates, and concrete pitfalls (label noise, annotator agreement/IAA, multiple-comparison/over-fitting traps, eval-vs-serve drift). What metrics + protocols should a small-data sports-segmentation leaderboard standardize on?

Deliverable: a structured, cited report with, per dimension, the 3?5 highest-leverage techniques that APPLY to our constrained setting (small data, local, license-clean, 1-D trajectory + cues + learned scorer), each with: what it is, why it fits our over-merge/small-data problem, the expected difficulty/payoff, and the citation. Call out which ideas are low-effort high-confidence wins vs research bets. Be explicit about anything unverified or license-incompatible.

Claim under review

"TemporalMaxer is a single-stage TAL detector built directly on the ActionFormer anchor-free head (same sequence-labeling decoder where every moment is classified plus onset/offset regression via Focal Loss + DIoU loss), with the ONLY change being that ActionFormer's self-attention temporal-context-modeling block is replaced by a parameter-free MaxPooling block (kernel 3, stride 2) between feature-pyramid levels."

****Source:**** <https://arxiv.org/abs/2303.09055> (primary)

****Supporting quote:**** "Similar to ActionFormer, TemporalMaxer follows a minimalistic design of sequence labeling where every moment is classified, and their corresponding action boundaries are regressed... The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block."

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

****refuted=true**** if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.

****refuted=false**** ONLY if: claim is well-supported, current, and source quality matches claim strength.

Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-06-16T03:24:31.387Z)

I'll verify this claim about TemporalMaxer's architecture by examining the primary source and checking the technical details.

3. assistant (2026-06-16T03:24:34.995Z)

Let me fetch the primary source (the TemporalMaxer arXiv paper) to verify the specific architectural claims.

4. user (2026-06-16T03:24:42.695Z)

TemporalMaxer Architecture Analysis

Based on the provided arXiv abstract and page content, I can only provide limited architectural details, as the full paper text is not included:

Available Information

****Parameter-free design:**** The abstract confirms that TemporalMaxer uses "a basic, parameter-free, and local region operating max-pooling block."

****Core replacement strategy:**** The work replaces sophisticated temporal context modeling. According to the abstract, it "minimizes long-term temporal context modeling while maximizing information from the extracted video clip features with a basic, parameter-free, and local region operating max-pooling block."

Information Not Available

The provided content does not include:

- Whether it's built on ActionFormer specifically
- Specific kernel size and stride parameters
- Details about the classification/regression heads
- Loss functions employed (Focal Loss, DIOU loss, etc.)
- Whether max-pooling is applied between feature pyramid levels

Recommendation

To answer questions (1), (3), (4), and (5) with accuracy, you would need to access the full PDF paper available at ``/pdf/2303.09055`` on arXiv, as the abstract alone doesn't provide these architectural implementation details.

5. assistant (2026-06-16T03:24:45.067Z)

The abstract confirms the parameter-free max-pooling block but not the finer details. Let me fetch the full paper PDF and search for corroborating sources.

6. user (2026-06-16T03:24:51.243Z)

I cannot find detailed information about TemporalMaxer architecture in the provided PDF content. The document appears to contain encoded/compressed PDF data that doesn't render readable text about TemporalMaxer's specific architectural components.

To accurately answer your questions about:

1. Whether it's built on ActionFormer
2. What components are replaced
3. MaxPooling kernel/stride specifications
4. Loss functions (Focal Loss, DIOU/GIOU)
5. Feature pyramid operations

I would need access to a readable version of the paper or document describing TemporalMaxer. The current source material doesn't contain extractable text addressing these architectural details.

[Binary content (application/pdf, 1.1MB) also saved to C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\tool-results\webfetch-1781579965172-5ov5x2.pdf]

7. assistant (2026-06-16T03:24:53.471Z)

The PDF was saved locally. Let me read it directly, and also search the web for corroborating descriptions in parallel.

8. user (2026-06-16T03:24:54.431Z)

pdftoppm failed: Command 'pdftoppm' not found or is in an unsafe location (current directory)

9. user (2026-06-16T03:25:01.514Z)

Web search results for query: "TemporalMaxer ActionFormer replace transformer self-attention max pooling block kernel stride"

Links: [{"title": "TemporalMaxer: Maximize Temporal Context with only Max Pooling", "url": "https://arxiv.org/pdf/2303.09055"}, {"title": "GitHub - f1brahim-tmu/TemporalMaxer: TemporalMaxer: Maximize Temporal Context with only Max Pooling for Temporal Action Localization · GitHub", "url": "https://github.com/f1brahim-tmu/TemporalMaxer"}, {"title": "TemporalMaxer Performance in the Face of Constraint: A Study in Temporal", "url": "https://repository.tudelft.nl/file/File_7430e57c-b269-4010-a020-376bf7180e18"}, {"title": "TemporalMaxer/README.md at master · TuanTNG/TemporalMaxer", "url": "https://github.com/TuanTNG/TemporalMaxer/blob/master/README.md"}, {"title": "Efficient Spatial-Temporal Focal Adapter with SSM for Temporal Action Detection This work was supported by JST CRONOS JPMJCS24K4.", "url": "https://arxiv.org/html/2604.09164"}, {"title": "Benchmarking Data Efficiency and Computational Efficiency of Temporal Action Localization Models", "url": "https://arxiv.org/pdf/2308.13082"}, {"title": "[2303.09055] TemporalMaxer: Maximize Temporal Context with only Max Pooling for Temporal Action Localization", "url": "https://arxiv.org/abs/2303.09055"}, {"title": "GitHub - zhenyingfang/Awesome-Temporal-Action-Detection-Temporal-Action-Proposal-Generation: Temporal Action Detection & Weakly Supervised Temporal Action Detection & Temporal Action Proposal Generation · GitHub", "url": "https://github.com/zhenyingfang/Awesome-Temporal-Action-Detection-Temporal-Action-Proposal-Generation"}]

Based on the search results, here's what I found about TemporalMaxer and its relationship to ActionFormer:

Overview

TemporalMaxer introduces a basic, parameter-free, and local region operating max-pooling block that minimizes long-term temporal context modeling while maximizing information from extracted video clip features.

Replacing Transformer Self-Attention

TemporalMaxer was motivated by PoolFormer, which replaces the computationally intensive and highly parameterized attention module with the most basic block in deep learning, the pooling layer. TemporalMaxer outperforms other state-of-the-art methods that utilize long-term temporal context modeling (TCM) such as self-attention on various TAL datasets while requiring significantly fewer parameters and computational resources.

Performance Compared to ActionFormer

TemporalMaxer improves the robust baseline ActionFormer by 2.4% mAP at tIoU=0.7, and 1.3% mAP on average. Additionally, TemporalMaxer outperforms ActionFormer with 2.8x fewer GMACs and 3x faster inference speed.

Max Pooling Block Specifications

Ablation studies with different kernel sizes for TemporalMaxer show that kernel size 3, 4, 5,

and 6 achieve average mAPs of 67.7, 67.1, 66.8, and 65.7 respectively, with the model achieving highest performance with kernel size 3.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

10. assistant (2026-06-16T03:25:04.755Z)

Good corroboration. The search confirms it's based on ActionFormer, replaces self-attention (motivated by PoolFormer), and uses kernel size 3 as optimal. Let me fetch the GitHub README and the HTML version of the paper to verify the precise details: the stride, the head/loss specifics, and "between feature pyramid levels."

11. user (2026-06-16T03:25:08.827Z)

The server returned HTTP 404 Not Found.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. `gh` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

12. user (2026-06-16T03:25:13.678Z)

Based on the README content provided, I can address some of your questions:

****Relationship to ActionFormer:****

The document indicates TemporalMaxer outperforms ActionFormer: "TemporalMaxer is effective at modeling temporal contexts, which outperforms the robust baseline, ActionFormer, with 2.8x fewer GMACs and 3x faster inference speed." However, it doesn't explicitly state whether TemporalMaxer builds upon ActionFormer's architecture.

****What it replaces:****

The paper replaces complex temporal context modeling blocks with a simpler approach: "TemporalMaxer, which minimizes long-term temporal context modeling while maximizing information from the extracted video clip features with a basic, parameter-free, and local region operating max-pooling block."

****Max-pooling specifications:****

The README does not provide the kernel size, stride, or other specific architectural parameters for the max-pooling layer.

****Classification/Regression heads and losses:****

The README does not mention details about the classification head, regression head, or loss functions (Focal Loss, DIOU/GIOU, etc.).

To find these architectural details, you would need to consult the full paper at <https://arxiv.org/pdf/2303.09055.pdf>, which isn't included in this README excerpt.

13. assistant (2026-06-16T03:25:16.652Z)

The HTML version doesn't exist (404, common for older papers). Let me convert the saved PDF to text directly so I can read the architecture and implementation sections verbatim, and verify the loss functions which are the part most likely to be an overreach.

14. user (2026-06-16T03:25:23.942Z)

LEN 51832

==KW ActionFormer count 24

==KW self-attention count 7

==KW Transformer count 16

==KW max pooling count 0

==KW Max Pooling count 14

==KW MaxPooling count 2

==KW kernel count 10

```
==KW stride count 6
==KW Focal count 2
==KW DIOU count 1
==KW GIOU count 0
==KW regress count 8
==KW sequence labeling count 3
==KW pyramid count 14
```

15. assistant (2026-06-16T03:25:26.913Z)

PDF text extracted successfully. Key finding: "DIOU count 1" and "GIOU count 0" ? so the paper mentions DIOU, not GIOU. Let me extract the precise passages around the load-bearing claims: the architecture replacement, kernel/stride, head, and loss functions.

16. user (2026-06-16T03:25:31.441Z)

----- around 'Focal'

ivation function. F_c and F_o are both 1D convolution. Note that all the weights of the decoder are shared between the different features in the multi-scale feature pyramidZ. Learning Objective . The model predicts $\hat{y}_t = (o_s, t, o_e, t, c, t)$ for every moment of the inputX. Following the baseline [60], the Focal Loss [31] and DIOU loss [64] are employed to supervise classification and regression outputs respectively. The overall loss function is defined as: $L_{total} = \sum_t (L_{cls} + 1 \cdot ct_{lreg}) / T + (7)$ where L_{reg} denotes regression loss and is applied only when the indicator function, $1 \cdot ct$, indicates that the current time step is a positive sample. $T +$ is the number of positive samples. L_{cls} isC way classification loss.

----- around 'DIOU'

F_c and F_o are both 1D convolution. Note that all the weights of the decoder are shared between the different features in the multi-scale feature pyramidZ. Learning Objective . The model predicts $\hat{y}_t = (o_s, t, o_e, t, c, t)$ for every moment of the inputX. Following the baseline [60], the Focal Loss [31] and DIOU loss [64] are employed to supervise classification and regression outputs respectively. The overall loss function is defined as: $L_{total} = \sum_t (L_{cls} + 1 \cdot ct_{lreg}) / T + (7)$ where L_{reg} denotes regression loss and is applied only when the indicator function, $1 \cdot ct$, indicates that the current time step is a positive sample. $T +$ is the number of positive samples. L_{cls} isC way classification loss. The loss fun

----- around 'sole variation'

p of the robust baseline model, ActionFormer [60]. This setup included various components such as decoder design d, non-maximum suppression (NMS) hyper-parameters in the post-processing stage, data augmentation, learning rate, feature extraction, and the number of feature pyramid level L. The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block. All experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent ablation will thorough analysis of the effects of varying kernel sizes. During training, the input feature length is kept constant at 2304, corresponding to approximately 5 minutes of video o

----- around 'substitution'

tionFormer [60]. This setup included various components such as decoder design d, non-maximum suppression (NMS) hyper-parameters in the post-processing stage, data augmentation, learning rate, feature extraction, and the number of feature pyramid level L. The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block. All experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent ablation will thorough analysis of the effects of varying kernel sizes. During training, the input feature length is kept constant at 2304, corresponding to approximately 5 minutes of video on both THUMOS14 and MultiTHUMOS data

----- around 'Max Pooling block'

ext, Graph [25] in G-TAD [54] and Transformers [51] in ActionFormer [60] to model long-term temporal contexts. Our proposed method, termed as TemporalMaxer, for the first time exploits the potentials of strong features from pretrained 3D CNN by only utilizing a basic, parameter-free, local operating Max Pooling block. Our proposed method, the simplest backbone ever for TAL, maintains only the most critical information on adjacent and local clip embeddings. Combined with the large receptive field of deep networks, the whole model outperforms other works by a large margin on various datasets. than employing exhausting long-term encoding process. The transformer architecture leading state-of-the-

----- around 'kernel size'

-parameters in the post-processing stage, data augmentation, learning rate, feature extraction, and the number of feature pyramid level L . The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block. All experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent ablation will thorough analysis of the effects of varying kernel sizes. During training, the input feature length is kept constant at 2304, corresponding to approximately 5 minutes of video on both THUMOS14 and MultiTHUMOS datasets, roughly 20 minutes on the EPIC-Kitchens 100 dataset, and approximately 45 minutes on MUSES. Additionally, Model

----- around 'stride'

volutional neural network layer, followed by Layer Normalization [3], and ReLU [1]. We simply assign $Z_1 = X_p$ as the first feature in Z . Finally, the multi-scale temporal feature pyramid Z is encoded by TemporalMaxer: $Z_l = \text{TemporalMaxer}(Z_{l-1})$. (4) Here: TemporalMaxer is Max Pooling and employed with stride 2, $Z_l \in \mathbb{R}^{T/2^{l-1} \times D}$, $2 \leq l \leq L$. It is worth noting that ActionFormer [60] employs Transformer [51] as a TCM block where each clip feature at the moment t represents a token, our proposed method adopts only Max Pooling [5] as TCM block. Decoder Design The decoder d learns to predict sequence labeling, $\hat{y} = \{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_T\}$, for every moment using multi-scale feature pyramid Z

----- around 'sequence labeling'

studies [60, 35, 63, 23] have employed Transformers as a long-term TCM. Our approach, TemporalMaxer, belongs to the single-stage TAL model that utilizes a state-of-the-art ActionFormer [60] as the baseline for comparison. Similar to ActionFormer, TemporalMaxer follows a minimalistic design of sequence labeling where every moment is classified, and their corresponding action boundaries are regressed. The main difference is that we avoid exhausting attention between clips from long-term timeframes which can unintentionally attend minute information among the crowd of similar frames, but keep the minute information in short-term manner. 3. Method 3.1. Problem Statement Temporal Actio

17. assistant (2026-06-16T03:25:44.860Z)

I now have the full verbatim text for every load-bearing part of the claim. Let me verify each component against the paper.

The claim has these components:

1. "single-stage TAL detector built directly on the ActionFormer anchor-free head" ? Paper: "belongs to the single-stage TAL model that utilizes a state-of-the-art ActionFormer [60] as the baseline" ?
2. "same sequence-labeling decoder where every moment is classified plus onset/offset regression" ? Paper: "Similar to ActionFormer, TemporalMaxer follows a minimalistic design of sequence labeling where every moment is classified, and their corresponding action boundaries are regressed" ?
3. "via Focal Loss + DIOU loss" ? Paper: "the Focal Loss [31] and DIOU loss [64] are employed to supervise classification and regression outputs respectively" ? (exactly correct ? DIOU, not GIOU)
4. "ONLY change being that ActionFormer's self-attention temporal-context-modeling block is replaced by a parameter-free MaxPooling block" ? Paper: "The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block" and "parameter-free... Max Pooling block" ?
5. "(kernel 3, stride 2)" ? Paper: "All experiments are conducted with a kernel size of 3 for all TCM blocks" + "Max Pooling and employed with stride 2" ?
6. "between feature pyramid levels" ? Paper: "the multi-scale temporal feature pyramid Z is encoded by TemporalMaxer: $Z_l = \text{TemporalMaxer}(Z_{l-1})$... Max Pooling and employed with stride 2, $Z_l \in \mathbb{R}^{T/2^{l-1} \times D}$ " ? (max pooling downsamples between pyramid levels)

Every single component of the claim is directly and verbatim supported by the primary source. Let me check the one minor nuance: the claim says "anchor-free head." The paper describes sequence-labeling where every moment is classified with boundary regression ? this IS the anchor-free formulation ActionFormer uses (no predefined anchors; per-moment onset/offset regression). This is accurate terminology.

One small precision note: the claim says "between feature pyramid levels" ? the paper applies max pooling (stride 2) to produce each successive pyramid level from the previous one, which is exactly "between levels." Also the claim's parenthetical "(same sequence-labeling decoder...)"

is accurate, though the decoder weights are shared and the architecture replaces the *encoder/TCM* block (not the decoder) ? which the claim correctly states (it says the self-attention TCM block is replaced, decoder stays the same).

The claim is precise, well-supported, current (2023 CVPR-W paper, still SOTA-relevant for efficient TAL), and sourced to the primary arXiv paper plus corroborated by the GitHub repo and an independent search. No contradicting evidence exists. The claim is NOT refuted.

18. user (2026-06-16T03:25:55.330Z)

Structured output provided successfully

19. assistant (2026-06-16T03:25:58.746Z)

Claim verified. Every load-bearing detail ? ActionFormer baseline, single-stage anchor-free sequence-labeling head (per-moment classification + boundary regression), the "sole variation" being the Transformer?parameter-free Max Pooling block swap, kernel size 3, stride 2, applied between feature-pyramid levels, and Focal Loss + DIoU loss (not GIoU) ? is verbatim-supported by the primary source arXiv:2303.09055 (extracted directly from the PDF) and corroborated by independent search and the official GitHub repo. **refuted=false**, high confidence.